Sri Lanka Institute of Information Technology
Department of Information Technology — Faculty of Computing

IT4020: Modern Topics in IT

Assignment 2

Microservices Architecture for a Real-World Business Domain

Business Domain	Online Food Delivery System
Tech Stack	Node.js · Express · MongoDB · Swagger
API Gateway	Custom Node.js Reverse Proxy
Submission Deadline	31 March 2026

Group Members

Name	Microservice Contribution
Ushan	Restaurant Service (Port 3001)
Malshan	Order Service (Port 3002)
Lashini	Customer Service (Port 3003)
Ridmi	Delivery Service (Port 3004)

1. Business Domain — Online Food Delivery System

The selected business domain for this group assignment is an Online Food Delivery System — a platform that connects customers with restaurants, manages orders, and coordinates delivery agents. This is a real-world, high-demand domain used by companies such as PickMe Food, Uber Eats, and DoorDash.

This domain was chosen because it naturally decomposes into clearly separated sub-domains, each of which can be developed as an independent microservice by an individual group member. Each service owns its own data, business logic, and API, making it an ideal use case for microservices architecture.

1.1 Why Microservices for This Domain?

- Each service (Restaurant, Order, Customer, Delivery) evolves independently.
- Teams can deploy updates to one service without affecting others.
- Specific services (e.g., Order) can be scaled during peak hours independently.
- Failures in one service (e.g., Delivery) are isolated and do not bring down the entire system.
- Each sub-domain has distinct data concerns and can use the most suitable database.

2. Microservices Identified

The following four microservices have been identified, one per group member. Each service is a standalone Node.js + Express application with its own MongoDB database, running on a dedicated port, and documented with Swagger UI.

Member	Microservice	Port	Database	Endpoints
Ushan	Restaurant Service	3001	restaurants_db	CRUD /restaurants, /menus
Malshan	Order Service	3002	orders_db	CRUD /orders, /cart
Lashini	Customer Service	3003	customers_db	CRUD /customers, /auth
Ridmi	Delivery Service	3004	delivery_db	CRUD /deliveries, /drivers

2.1 Restaurant Service — Ushan (Port 3001)

Responsibility
Manages restaurant profiles, their menus, and food item listings. This service is the entry point for browsing available restaurants and their offerings.

Key Endpoints:

- GET /api/restaurants — List all restaurants
- GET /api/restaurants/:id — Get restaurant by ID
- POST /api/restaurants — Create a new restaurant
- PUT /api/restaurants/:id — Update restaurant details
- DELETE /api/restaurants/:id — Remove a restaurant
- GET /api/restaurants/:id/menu — Get menu for a restaurant
- POST /api/restaurants/:id/menu — Add menu item

MongoDB Schema: Restaurant { name, address, cuisine, rating, menu: [{ itemName, price, description, available }] }

2.2 Order Service — Malshan (Port 3002)

Responsibility
Handles the lifecycle of a customer order: creating orders, tracking status (PENDING → CONFIRMED → PREPARING → READY → DELIVERED), and managing cart operations before checkout.

Key Endpoints:

- GET /api/orders — Get all orders
- GET /api/orders/:id — Get order by ID
- POST /api/orders — Place a new order
- PUT /api/orders/:id/status — Update order status
- DELETE /api/orders/:id — Cancel an order
- GET /api/cart/:customerId — Get cart for customer
- POST /api/cart — Add item to cart

MongoDB Schema: Order { customerId, restaurantId, items: [{ menuItemId, qty, price }], totalAmount, status, createdAt }

2.3 Customer Service — Lashini (Port 3003)

Responsibility
Manages customer accounts, registration, login (JWT-based authentication), and profile management. This service acts as the identity provider for the platform.

Key Endpoints:

- POST /api/customers/register — Register new customer
- POST /api/customers/login — Login and receive JWT token
- GET /api/customers/:id — Get customer profile
- PUT /api/customers/:id — Update customer profile
- DELETE /api/customers/:id — Delete account
- GET /api/customers/:id/orders — Get order history for customer

MongoDB Schema: Customer { name, email, passwordHash, phone, address, createdAt }

2.4 Delivery Service — Ridmi (Port 3004)

Responsibility
Manages delivery agents (drivers), tracks assigned deliveries, and updates real-time delivery status from pickup to drop-off. Coordinates with the Order Service through the API Gateway.

Key Endpoints:

- GET /api/deliveries — List all deliveries
- GET /api/deliveries/:id — Get delivery by ID
- POST /api/deliveries — Create a delivery assignment
- PUT /api/deliveries/:id/status — Update delivery status
- GET /api/drivers — List all delivery drivers
- POST /api/drivers — Register a new driver
- PUT /api/drivers/:id — Update driver availability

MongoDB Schema: Delivery { orderId, driverId, pickupLocation, dropLocation, status, estimatedTime }

3. Technology Stack

The following technology stack has been selected based on the requirements of the assignment, scalability considerations, and compatibility with the team's existing familiarity with JavaScript-based tools.

Category	Tool / Technology	Purpose
Runtime	Node.js 20 LTS	JavaScript server-side runtime for all microservices
Framework	Express.js 4.x	REST API framework for routing and middleware
Database	MongoDB + Mongoose	NoSQL document database (one DB per service)
API Gateway	Custom Node.js + http-proxy-middleware	Single entry point, port unification, routing
API Docs	Swagger UI (swagger-jsdoc)	Auto-generated API documentation at /api-docs
Auth (Lashini)	JWT (jsonwebtoken)	Stateless authentication tokens
Frontend	Next.js 14 (App Router)	Optional — UI for testing the gateway endpoints
Dev Tools	Nodemon + dotenv	Hot reload and environment variable management
Package Mgmt	npm / package.json	Dependency management per microservice

Additionally recommended tools for the project:

- Postman — for manually testing all API endpoints during development
- MongoDB Atlas — free cloud MongoDB hosting, or use local MongoDB Community Edition
- VS Code — recommended editor with REST Client extension
- Git + GitHub — version control (one repository per microservice, or a monorepo)

4. Project Folder Structure

The solution follows a monorepo structure where all microservices and the API Gateway co-exist in a single root repository. Each service is completely self-contained with its own node_modules, package.json, and .env file.

```
food-delivery-microservices/
│
├── api-gateway/                  ← API Gateway (Port 5000)
│   ├── index.js
│   ├── package.json
│   └── .env
│
├── restaurant-service/           ← Ushan (Port 3001)
│   ├── src/
│   │   ├── models/Restaurant.js
│   │   ├── routes/restaurantRoutes.js
│   │   ├── controllers/restaurantController.js
│   │   └── swagger.js
│   ├── index.js
│   ├── package.json
│   └── .env
│
├── order-service/                ← Malshan (Port 3002)
│   ├── src/
│   │   ├── models/Order.js
│   │   ├── routes/orderRoutes.js
│   │   ├── controllers/orderController.js
│   │   └── swagger.js
│   ├── index.js
│   ├── package.json
│   └── .env
│
├── customer-service/             ← Lashini (Port 3003)
│   ├── src/
│   │   ├── models/Customer.js
│   │   ├── routes/customerRoutes.js
│   │   ├── controllers/customerController.js
│   │   └── swagger.js
│   ├── index.js
│   ├── package.json
│   └── .env
│
├── delivery-service/             ← Ridmi (Port 3004)
│   ├── src/
│   │   ├── models/Delivery.js
│   │   ├── routes/deliveryRoutes.js
│   │   ├── controllers/deliveryController.js
│   │   └── swagger.js
│   ├── index.js
│   ├── package.json
│   └── .env
│
└── README.md
```

5. API Gateway — Eliminating Multiple Ports

Without an API gateway, a frontend or client must know and maintain the port number of every individual microservice (e.g., 3001, 3002, 3003, 3004). This creates tight coupling and complexity. The API Gateway solves this by acting as a single reverse proxy running on Port 5000.

Without API Gateway (Bad)	With API Gateway (Good)
• Client calls :3001 for restaurants • Client calls :3002 for orders • Client calls :3003 for customers • Client calls :3004 for delivery • 4 different ports to manage	• All calls go to :5000 only • :5000/restaurants → :3001 • :5000/orders → :3002 • :5000/customers → :3003 • :5000/delivery → :3004

5.1 API Gateway Implementation Code

Below is the complete implementation of the API Gateway (api-gateway/index.js):

```
// api-gateway/index.js
const express = require('express');
const { createProxyMiddleware } = require('http-proxy-middleware');
const app = express();

// Route: /restaurants → Restaurant Service on port 3001
app.use('/restaurants', createProxyMiddleware({
  target: 'http://localhost:3001',
  changeOrigin: true,
  pathRewrite: { '^/restaurants': '/api/restaurants' }
}));

// Route: /orders → Order Service on port 3002
app.use('/orders', createProxyMiddleware({
  target: 'http://localhost:3002',
  changeOrigin: true,
  pathRewrite: { '^/orders': '/api/orders' }
}));

// Route: /customers → Customer Service on port 3003
app.use('/customers', createProxyMiddleware({
  target: 'http://localhost:3003',
  changeOrigin: true,
  pathRewrite: { '^/customers': '/api/customers' }
}));

// Route: /delivery → Delivery Service on port 3004
app.use('/delivery', createProxyMiddleware({
  target: 'http://localhost:3004',
  changeOrigin: true,
  pathRewrite: { '^/delivery': '/api/deliveries' }
}));
```

```
app.listen(5000, () => {
  console.log('API Gateway running on http://localhost:5000');
});
```

Install dependency for the gateway:

```
cd api-gateway && npm init -y && npm install express http-proxy-middleware
```

6. Step-by-Step Implementation Guide (Per Service)

All four microservices follow the same pattern. Below is the complete setup and code for each service. Replace [SERVICE] with restaurant, order, customer, or delivery as appropriate.

6.1 Initial Setup (All Members — Run Once Per Service)

Step 1 — Initialise the project:

```
mkdir restaurant-service && cd restaurant-service
npm init -y
npm install express mongoose dotenv swagger-jsdoc swagger-ui-express
npm install --save-dev nodemon
```

Step 2 — Create .env file:

```
PORT=3001
MONGO_URI=mongodb://localhost:27017/restaurants_db
```

Step 3 — Update package.json scripts:

```
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}
```

6.2 Express + Swagger Bootstrap (index.js)

Create index.js at the root of each service directory:

```
const express = require('express');
const mongoose = require('mongoose');
const swaggerUi = require('swagger-ui-express');
const swaggerSpec = require('./src/swagger');
require('dotenv').config();

const app = express();
app.use(express.json());

// Swagger UI — accessible at /api-docs
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));

// Mount routes
app.use('/api/restaurants', require('./src/routes/restaurantRoutes'));

// Connect to MongoDB
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.error(err));

const PORT = process.env.PORT || 3001;
app.listen(PORT, () => console.log(`Restaurant Service on port ${PORT}`));
```

6.3 Swagger Configuration (src/swagger.js)

```
const swaggerJsDoc = require('swagger-jsdoc');

const options = {
  definition: {
    openapi: '3.0.0',
    info: {
      title: 'Restaurant Service API',
      version: '1.0.0',
      description: 'API for managing restaurants and menus'
    },
    servers: [{ url: 'http://localhost:3001' }]
  },
  apis: ['./src/routes/*.js']  // JSDoc comments in routes are parsed
};

module.exports = swaggerJsDoc(options);
```

6.4 Swagger JSDoc Example in Routes

Add JSDoc comments above each route handler for auto-generated Swagger docs:

```
/**
 * @swagger
 * /api/restaurants:
 *   get:
 *     summary: Retrieve all restaurants
 *     tags: [Restaurants]
 *     responses:
 *       200:
 *         description: List of all restaurants
 */
router.get('/', restaurantController.getAll);
```

6.5 Swagger URLs — Direct vs. Via API Gateway

Each service exposes its own Swagger UI at the native port. The API Gateway proxies API calls, but for Swagger UI you access it directly per service. The table below shows both access methods required for the assignment screenshots.

Service	Native Swagger URL	Via API Gateway (API)
Restaurant (Ushan)	`http://localhost:3001/api-docs`	`http://localhost:5000/restaurants`
Order (Malshan)	`http://localhost:3002/api-docs`	`http://localhost:5000/orders`
Customer (Lashini)	`http://localhost:3003/api-docs`	`http://localhost:5000/customers`
Delivery (Ridmi)	`http://localhost:3004/api-docs`	`http://localhost:5000/delivery`

7. Running the Full Solution

Follow these steps in order to start all microservices and the API Gateway:

7.1 Prerequisites

- Node.js 20+ installed (node -v to verify)
- MongoDB running locally on port 27017 OR a MongoDB Atlas connection string
- All npm install commands completed inside each service folder

7.2 Start Each Service (4 Separate Terminals)

Terminal 1 — Restaurant Service (Ushan):

```
cd restaurant-service && npm run dev
```

Terminal 2 — Order Service (Malshan):

```
cd order-service && npm run dev
```

Terminal 3 — Customer Service (Lashini):

```
cd customer-service && npm run dev
```

Terminal 4 — Delivery Service (Ridmi):

```
cd delivery-service && npm run dev
```

7.3 Start the API Gateway

Terminal 5 — API Gateway:

```
cd api-gateway && npm run dev
```

Once all services are running, verify them by opening in your browser:

- http://localhost:3001/api-docs — Restaurant Swagger
- http://localhost:3002/api-docs — Order Swagger
- http://localhost:3003/api-docs — Customer Swagger
- http://localhost:3004/api-docs — Delivery Swagger
- http://localhost:5000/restaurants — Via API Gateway

8. Slide Deck Outline

The following slide structure is recommended for the submission. Each slide maps directly to the assignment's evaluation criteria.

Slide	Title	Content
1	Title Slide	Assignment title, module code, group member names and contributions, date
2	Business Domain	Online Food Delivery System overview, why microservices fit this domain
3	Architecture Diagram	Visual diagram of all 4 services + API Gateway + MongoDB databases
4	Microservice 1 — Ushan	Restaurant Service: responsibilities, endpoints, MongoDB schema, Swagger screenshot
5	Microservice 2 — Malshan	Order Service: responsibilities, endpoints, MongoDB schema, Swagger screenshot
6	Microservice 3 — Lashini	Customer Service: responsibilities, endpoints, JWT auth, Swagger screenshot
7	Microservice 4 — Ridmi	Delivery Service: responsibilities, endpoints, MongoDB schema, Swagger screenshot
8	API Gateway	Problem it solves, routing table, code snippet, before/after comparison
9	Demo Screenshots	Side-by-side: native swagger URL vs. API gateway URL for each service
10	Folder Structure	Screenshot of project directory in VS Code
11	Tech Stack	Icons/logos of Node.js, Express, MongoDB, Swagger, Next.js
12	Conclusion	Learnings, challenges, individual contributions summary

9. Submission Checklist

Use the following checklist before submitting to ensure all evaluation criteria are covered:

✓	Criterion
☐	Each group member has built their own dedicated microservice (one per person)
☐	All microservices run without build or runtime errors
☐	Each service has its own MongoDB database (not shared)
☐	Swagger is accessible at /api-docs on each service's native port
☐	API Gateway routes all services through a single port (5000)
☐	Screenshot of each Swagger UI accessed via native port included in slides
☐	Screenshot of API gateway endpoints responding correctly included in slides
☐	Folder structure clearly shows separation of services and gateway
☐	Slide deck includes each member's name and their specific contribution
☐	Slide deck is of industry-level presentation quality
☐	Each member can explain the entire architecture and their own service

Plagiarism Notice

All code submitted must be written by the respective group member. Using open-source references for learning is acceptable, but directly copying and submitting code written by others will result in zero marks for the entire group, per SLIIT's plagiarism policy.